

DAS CLAUDE-HANDBUCH

Stufe Eins

Von einzelnen Aufgaben zum Bauen eigener Lösungen.

Jetzt mit Diagrammen, durchgearbeiteten Beispielen und Schritt-für-Schritt-Vorgehen.

Prompting · Projects · Skills · Connectors · Daten & DSGVO · Quellcode & Git · Code · Agenten

Die Fortsetzung von Stufe Null — für den Moment, in dem Ihnen die Grundlagen leichtfallen.

Von Hans Tandt

Ardennen, Wallonie, Belgien

Version 2.2 — September 2026

So nutzen Sie dieses Buch

Dies ist Stufe Eins. Sie setzt voraus, dass Sie Stufe Null durchgearbeitet haben: Sie können Claude einrichten, eine Aufgabe in Cowork ausführen, Ihre Dateien finden und eine Sitzung verwalten. Wenn Ihnen das noch nicht vertraut ist, lesen Sie zuerst Stufe Null — dieses Buch baut direkt darauf auf.

Noch nie eine KI benutzt? Beginnen Sie mit der zweiseitigen Anleitung **Hier beginnen**, danach mit Stufe Null.

Welches Abo brauchen Sie?

Der Großteil von Stufe Eins nutzt Cowork, Code, Connectors und geplante Aufgaben. Dafür brauchen Sie ein bezahltes Abo (siehe Stufe Null, Kapitel 3).

Kapitel 1 (Prompting) und Kapitel 5 (Daten und DSGVO) sind mit jedem Abo nützlich.

In Stufe Eins geht es um den Sprung von einzelnen Aufgaben hin zu Lösungen, die dauerhaft funktionieren: besseres Prompting, wiederverwendbare Skills, verbundene Tools, Daten, denen Sie vertrauen können, gemeinsamer Quellcode, ein behutsamer Einstieg in Code und Ihr erster kleiner automatisierter Agent.

Drei Wege — lesen Sie, was Sie brauchen

Alles ist neu für Sie: Lesen Sie der Reihe nach.

Sie kennen die Grundlagen und wollen bessere Ergebnisse: Kapitel 1–5 (Prompting, Projects, Skills, Connectors, Daten & DSGVO).

Sie brauchen jetzt Teamarbeit: Springen Sie zu Kapitel 7–9 (wo der Quellcode liegt, Git, Arbeiten im Team) — sie stehen für sich allein.

Dieses Buch ist eine Momentaufnahme — Claude ändert sich schnell

Alles hier gibt den Stand von Claude im August 2026 wieder: die Modellfamilien Opus, Sonnet, Haiku und Fable, die Desktop-App Cowork und die gemeinsame Startseite, die Chat, Cowork und Ihre Projects in einer Seitenleiste vereint. Anthropic liefert Updates sehr schnell aus — oft in Tagen, nicht in Monaten. Ein Bildschirm, ein Menüname oder eine Modellnummer kann sich also schon geändert haben. Wenn das, was Sie sehen, nicht zur Seite passt, vertrauen Sie der App und claude.com. Die Ideen und Gewohnheiten hier gelten auch dann, wenn die Schaltflächen wandern.

Chat und Cowork werden eine App

Seit Ende September 2026 führt Anthropic Chat und Cowork zu einer einzigen Claude-App zusammen, zuerst bei Pro und Max. Wo dieser Leitfaden Cowork sagt, sehen Sie vielleicht einfach Claude. Die Ideen und Gewohnheiten bleiben gleich.

Was ist neu in Version 2.2

Neu: eine kurze Erklärung jedes Fachworts dort, wo es zum ersten Mal vorkommt; ein nach Themen gegliedertes Glossar; ein Hinweis, welches Abo Sie brauchen. Aktualisiert: Persönliche Faustregeln sind jetzt als Erfahrung gekennzeichnet, nicht als Tatsache.

Inhaltsverzeichnis

So nutzen Sie dieses Buch.....	2
Stufe Null auf einer Seite.....	4
1. Prompting, das wirklich funktioniert.....	5
1.1 Kontext in Aktion — vorher / nachher.....	6
1.2 Die Anti-Stil-Datei.....	6
1.3 Erst zeigen lassen, dann korrigieren.....	6
1.4 Die Antwort prüfen — was und wie.....	6
2. Projects im Detail.....	7
2.1 Ein Project pro Ergebnis.....	7
2.2 Was in die Anweisungen gehört.....	7
2.3 Vorgehen — ein nützliches Project in 5 Minuten.....	7
3. Ihren ersten Skill bauen.....	8
3.1 Was ein Skill wirklich ist.....	8
3.2 Vorgehen — einen Skill erstellen und testen.....	8
4. Connectors — Claude mit Ihren anderen Tools verbinden.....	9
4.1 Verbinden Sie nur, was die Aufgabe braucht.....	9
4.2 Die drei Arten von Connectors.....	9
4.3 Ein erster sicherer Einsatz.....	9
5. Daten, Sicherheit und DSGVO, bevor Sie verbinden.....	10
5.1 Die Daten-Checkliste vor dem Verbinden.....	10
5.2 Die DSGVO in einfachen Worten.....	10
5.3 Nachvollziehen und die Kontrolle behalten.....	11
6. Projekte parallel ausführen.....	12
6.1 Was gleichzeitig laufen kann.....	12
6.2 Die zwei echten Kosten.....	12
7. Wo der Quellcode Ihres Projekts liegt.....	13
7.1 Eine einfache Faustregel.....	13
7.2 Halten Sie eine ordentliche Struktur.....	13
8. Eine behutsame Einführung in Git.....	14
8.1 Was Git wirklich tut.....	14
8.2 Der tägliche Ablauf.....	14
8.3 Workshop — Ihr erstes Repo in 10 Minuten.....	14
9. Im Team am selben Projekt arbeiten.....	16
9.1 Dokumente und Code.....	16
9.2 Der Git-Ablauf im Team.....	16
10. Ein behutsamer erster Schritt in Code.....	17
10.1 Beschreiben statt tippen.....	17
10.2 CLAUDE.md — das Gehirn des Projekts.....	17
10.3 Berechtigungen — drei Stufen, nicht zwei.....	17
10.4 Erst planen, dann bauen.....	17
10.5 Vibe coding — und seine Grenzen.....	17
10.6 Claude Design — Oberflächen ohne Code.....	17
11. Ihr erster kleiner Agent.....	18
11.1 Was einen Agenten ausmacht.....	18
11.2 Ein vollständiges Beispiel — ein Morgenbriefing.....	18
11.3 Langsam ausbauen.....	18
12. Werkzeuge und Apps zum Installieren.....	19
13. Wartbar bleiben.....	20
Glossar der neuen Begriffe.....	21
Wie es weitergeht.....	23
Die API — der Schritt über dieses Buch hinaus.....	23

Stufe Null auf einer Seite

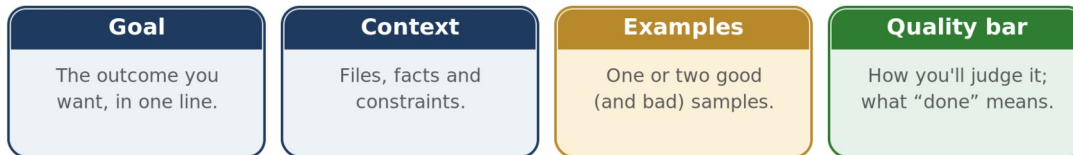
Bevor wir etwas bauen: die fünf Ideen, auf denen hier alles beruht.

1. **Wählen Sie das richtige Gesicht.** Chat zum Nachdenken, Cowork zum Erstellen von Dateien, Code zum Bauen von Software, Projects zum Merken, Connectors zum Erreichen anderer Apps.
2. **Kontext schlägt Raffinesse.** Was Claude sehen kann, bevor es antwortet, zählt mehr als die Formulierung des Prompts.
3. **Lassen Sie Claude zuerst fragen, dann prüfen Sie.** „Stell mir zuerst deine Fragen.“ Und vertrauen Sie nie einer selbstsicheren Antwort, ohne sie zu prüfen.
4. **Eine Datei ist erst in Ihrem Ordner echt.** Bis sie dort gespeichert ist, liegt sie in der Sandbox oder im Scratchpad und kann verschwinden.
5. **Halten Sie Sitzungen klein.** Beginnen Sie für jedes Thema neu. Stützen Sie sich auf Sitzungsnotizen und das Gedächtnis, damit Sie nie alles neu erklären müssen.

1. Prompting, das wirklich funktioniert

Stufe Null sagte: Kontext schlägt Raffinesse. Stufe Eins macht das praktisch. Eine gute Anfrage hat vier Teile, und keiner davon ist ein Zauberwort.

A good request has four parts (not a magic word)



Give the destination and the standards — not a long list of steps.

Teil	Was er bedeutet	Beispiel
Ziel	Das Ergebnis, das Sie wollen, in einem Satz.	„Entwirf ein einseitiges Angebotsschreiben.“
Kontext	Die Fakten, Dateien und Vorgaben, die Claude braucht.	„Nutze die angehängte Vorlage und unsere Standardbedingungen.“
Beispiele	Ein oder zwei Muster für gute (und schlechte) Ergebnisse.	„Triff den Ton dieses früheren Schreibens; vermeide diese Formulierung.“
Qualitätsmaßstab	Wie Sie es beurteilen; wie „fertig“ aussieht.	„Förmlich, unter 250 Wörtern, keine Modewörter.“

1.1 Kontext in Aktion — vorher / nachher

Gleiche Aufgabe, zwei Prompts. **Vorher:** „Schreib eine E-Mail an einen Lieferanten wegen der verspäteten Bestellung.“ Sie bekommen eine allgemeine, farblose E-Mail. **Nachher:** „Schreib eine E-Mail an unseren Lieferanten Dupont wegen Bestellung 4471, 9 Tage im Verzug. Bestimmt, aber höflich, bitte um einen neuen Liefertermin und eine Teilgutschrift, unter 120 Wörtern, unterschreib in meinem Namen.“ Sie bekommen etwas, das Sie fast so abschicken können. Der Unterschied ist nicht Raffinesse — es ist Kontext.

1.2 Die Anti-Stil-Datei

Der schnellste Weg gegen Texte, die nach KI klingen: Schreiben Sie auf, was Sie nie wollen. Eine kurze „Anti-Stil“-Datei — Wörter, die Sie nie verwenden, Einstiege, die Sie hassen, Muster, die Sie ablehnen — bewirkt mehr als jede positive Anweisung. Legen Sie sie in Ihren Ordner ABOUT ME, damit jede Sitzung sie liest.

1.3 Erst zeigen lassen, dann korrigieren

Versuchen Sie nicht, wichtige Arbeit im ersten Anlauf fertig zu bekommen. Bitten Sie um einen Entwurf, reagieren Sie darauf und lassen Sie Claude nachbessern. Nach meiner Erfahrung schlagen drei schnelle Runden „gut, aber kürzer und weniger förmlich“ eine halbe Stunde Feilen am perfekten ersten Prompt.

1.4 Die Antwort prüfen — was und wie

Claude klingt bei falschen Antworten genauso sicher wie bei richtigen. Lassen Sie Web Search eingeschaltet und bitten Sie Claude, sich selbst zu prüfen: „*Prüf deine Antwort, bevor ich ihr vertraue; überprüfe die Fakten und sag mir, wo du unsicher bist.*“ Bei allem, was wichtig ist, prüfen Sie zusätzlich an der Quelle.

2. Projects im Detail

Ein Project ist ein Ordner mit Chats, die Anweisungen, Dateien und Gedächtnis teilen. So legen Sie eines an, das Zeit spart, statt zur Rumpelkammer zu werden. Seit Mitte 2026 liegen Ihre Projects in der linken Seitenleiste der gemeinsamen Startseite, geteilt zwischen Chat und Cowork. Heften Sie die an, die Sie nutzen, damit Sie sie schnell finden.

2.1 Ein Project pro Ergebnis

Der häufige Fehler ist ein einziges riesiges Project für „alles“. Die Lösung: ein Project pro wiederkehrendem Ergebnis — ein Wochenbericht, eine Art von Angebot, ein bestimmtes Tool. Jedes bekommt nur die Dateien, die es braucht, und Anweisungen für genau diese eine Aufgabe.

2.2 Was in die Anweisungen gehört

Project-Anweisungen sind keine Dokumentation. Sie sind das Minimum, das verhindert, dass Claude abdriftet. Nach meiner Erfahrung reichen fünf bis acht Zeilen:

- Was dieses Ergebnis ist und für wen es gedacht ist.
- Der Ton und der Qualitätsmaßstab.
- Was immer zu tun ist und was nie.
- Wo das Ergebnis gespeichert wird und in welchem Format.

Dateien sind nicht umsonst

Mehr Dateien bedeuten nicht bessere Antworten — sie bedeuten mehr Rauschen, das gefiltert werden muss. Laden Sie Beispiele guter Arbeit und die Vorlagen hoch, die Sie wiederverwenden, nicht Ihr ganzes Archiv. Nach meiner Erfahrung liegt es oft an zu vielen Dateien, nicht an zu wenigen, wenn eine Antwort abdriftet.

2.3 Vorgehen — ein nützliches Project in 5 Minuten

1. Benennen Sie es nach einem Ergebnis (z. B. „Wöchentlicher Betriebsbericht“).
2. Fügen Sie nur die Vorlagen dieses Ergebnisses und 1–2 gute frühere Beispiele hinzu.
3. Schreiben Sie 5–8 Zeilen Anweisungen (was / für wen / Ton / wo speichern / wann nachfragen).
4. Testen Sie mit einer echten Aufgabe. Lassen Sie Claude sich selbst kritisieren und schärfen Sie die Anweisungen nach.

3. Ihren ersten Skill bauen

Ein Skill ist Ihre Standardmethode für eine Aufgabe, so aufgeschrieben, dass Claude sie jedes Mal gleich ausführt. Wenn Sie schon einmal eine Checkliste geschrieben haben, können Sie einen Skill bauen — ganz ohne Code.

3.1 Was ein Skill wirklich ist

Ein Skill ist eine kleine Textdatei namens `SKILL.md`. Oben steht eine kurze Beschreibung, darunter die vollständigen Anweisungen. Nur die Beschreibung liegt in Claudes Gedächtnis; der Rest wird geladen, wenn eine Aufgabe passt. Deshalb können Sie viele Skills haben, ohne dass etwas langsamer wird.

3.2 Vorgehen — einen Skill erstellen und testen

1. **Erledigen Sie die Aufgabe einmal von Hand** mit Claude und achten Sie auf die Schritte und die Fehler.
2. **Ernten Sie das Ergebnis:** „Mach aus dem, was wir gerade getan haben, einen wiederverwendbaren Skill. Schreib die Schritte als klare Anweisungen, füge einen kurzen Abschnitt ‚Stolperfallen‘ hinzu und halte die Beschreibung auf einer Zeile, damit er bei den richtigen Anfragen auslöst.“
3. **Testen Sie an einem neuen Beispiel**, das etwas anders ist, und beheben Sie, was der Skill übersehen hat.
4. **Speichern Sie ihn** in der App unter Customize, dann Skills. Ein Skill wird nur besser, wenn seine Datei bearbeitet wird — er lernt nicht von selbst.

Der schnellste Weg zum Skill: Nehmen Sie sich dabei auf

Seit Mitte 2026 hat Cowork im +-Menü der Desktop-App die Option „Record a skill“. Statt die Schritte aufzuschreiben, nehmen Sie Ihren Bildschirm auf und erklären die Aufgabe einmal laut, während Sie sie erledigen. Claude macht aus der Vorführung den Skill. Das ist der bequemste Weg für eine Aufgabe, die sich leichter zeigen als beschreiben lässt — besonders für Browser-Aufgaben ohne Connector. Nehmen Sie keine Passwörter oder privaten Daten auf. Verfügbar mit Pro, Max und Team.

4. Connectors — Claude mit Ihren anderen Tools verbinden

Ein Connector ist eine sichere Brücke zwischen Claude und einer anderen App — E-Mail, Kalender, ein Cloud-Speicher, ein Projekt-Tracker. Ist einer verbunden, kann Claude Daten außerhalb Ihres Ordners lesen und damit arbeiten.

4.1 Verbinden Sie nur, was die Aufgabe braucht

Der erste Impuls ist, alles zu verbinden. Widerstehen Sie ihm. Jeder Connector bringt Rauschen und verbraucht Kontext. Eine Aufgabe für das Morgen-Briefing braucht Ihre E-Mails und Ihren Kalender — nicht Ihre ganze Werkzeugkette. Schalten Sie Connectors aus, wenn ein Ablauf sie nicht nutzt.

4.2 Die drei Arten von Connectors

Connectors laufen auf einem gemeinsamen Standard namens **MCP** (Model Context Protocol). Sie finden und fügen sie in den Connectors-Einstellungen hinzu (das „+“ in einem Chat oder Settings, Connectors). Es gibt drei Arten, und der Unterschied ist für das Vertrauen wichtig:

- **Die eingebauten Integrationen von Anthropic** – aus erster Hand, von Anthropic selbst entwickelt (Gmail, Drive, GitHub, Slack...).
- **Connectors aus dem Verzeichnis** – von Drittanbietern (anderen Firmen) entwickelt, aber vor der Aufnahme von Anthropic geprüft.
- **Eigene MCP-Connectors, die Sie selbst hinzufügen** – diese hat niemand geprüft. Behandeln Sie sie mit derselben Vorsicht wie jede neue App, die Sie installieren.

4.3 Ein erster sicherer Einsatz

Beginnen Sie nur lesend (Claude darf schauen, aber nichts senden oder ändern). Bitten Sie Claude, die ungelesenen E-Mails von heute zusammenzufassen oder die Termine dieser Woche aufzulisten, bevor Sie es etwas senden oder ändern lassen. Bauen Sie Vertrauen bei Aufgaben auf, bei denen ein Fehler harmlos ist, und erweitern Sie dann schrittweise.

5. Daten, Sicherheit und DSGVO, bevor Sie verbinden

Sobald Claude echte Geschäftsdaten berührt — über einen Connector, ein eingefügtes Dokument oder einen Ordner –, liegt der Datenschutz in Ihrer Verantwortung. Dieses Kapitel schafft praktisches Bewusstsein und ist keine Rechtsberatung. Fragen Sie im Zweifel Ihre(n) Datenschutzbeauftragte(n) (die Person in Ihrer Organisation, die für personenbezogene Daten zuständig ist).

5.1 Die Daten-Checkliste vor dem Verbinden

1. Welche Daten wird Claude sehen, und identifiziert irgendetwas davon eine Person?
2. Habe ich einen Grund (eine Rechtsgrundlage), sie auf diese Weise zu verarbeiten?
3. Ist es das Minimum, das die Aufgabe braucht, oder teile ich zu viel?
4. Könnte ich die Daten anonymisieren oder schwärzen (Namen und Angaben entfernen, die Personen identifizieren) und trotzdem das Ergebnis bekommen?

5.2 Die DSGVO in einfachen Worten

Die DSGVO der EU regelt personenbezogene Daten — alles, was eine lebende Person identifiziert (Namen, E-Mail-Adressen, Mitarbeiter- oder Kundendaten, eine Unfallakte). Einige Grundsätze decken die meisten Alltagsfälle ab:

- **Haben Sie einen Grund.** Verarbeiten Sie personenbezogene Daten nur für einen klaren, legitimen Zweck.
- **Nutzen Sie das Minimum.** Geben Sie nicht mehr persönliche Angaben ein, als die Aufgabe braucht.
- **Bleiben Sie beim Zweck.** Verwenden Sie Daten nicht für etwas, womit die Betroffenen nicht rechnen würden.
- **Schützen und löschen.** Bewahren Sie die Daten sicher auf und nicht länger als nötig.

Wohin gehen die Daten? (Und kann ich sie lokal halten?)

Chat, Cowork und Code senden Ihre Inhalte zur Verarbeitung an die Server von Anthropic — Claude läuft nicht auf Ihrem Computer, und keine Einstellung hält Ihre Daten lokal. Sie KÖNNEN abschalten, dass Ihre Unterhaltungen zum Trainieren der Modelle genutzt werden (Settings, Privacy). Das verkürzt auch die Aufbewahrung (wie lange Ihre Daten gespeichert werden). Team-, Enterprise- und API-Konten trainieren standardmäßig nicht mit Ihren Daten und bieten stärkere Kontrolle über die Aufbewahrung. Für eine wirklich lokale Offline-KI bräuchten Sie ein anderes, selbst betriebenes Modell — und das ist nicht Claude.

5.3 Nachvollziehen und die Kontrolle behalten

Führen Sie bei allem Sensiblen eine einfache Aufzeichnung darüber, was Sie eingegeben haben und warum. Nutzen Sie bevorzugt die Ampelregel aus Stufe Null (grün / orange / rot) und sorgen Sie dafür, dass ein Mensch prüft, bevor etwas gesendet oder umgesetzt wird. Wenn ein Connector handeln kann (E-Mails senden, Datensätze ändern), starten Sie ihn nur lesend, bis Sie ihm vertrauen.

Keine Rechtsberatung

Dieses Kapitel vermittelt allgemeines Bewusstsein, damit Sie offensichtliche Fehler vermeiden. Die Pflichten aus der DSGVO hängen von Ihrer Rolle und Ihrer Organisation ab. Klären Sie die Einzelheiten mit Ihrer/Ihrem Datenschutzbeauftragten, bevor Sie personenbezogene Daten in großem Umfang verarbeiten.

6. Projekte parallel ausführen

Sie sind nicht auf eine Sache gleichzeitig beschränkt. Sie können in Cowork ein Dokument schreiben, während Claude in Code eine App repariert, und mehrere Cowork-Aufgaben gleichzeitig voranbringen. Die Tools laufen unabhängig voneinander — die Grenze sind Sie und Ihr Kontingent.

6.1 Was gleichzeitig laufen kann

- **Cowork und Code zusammen.** Zwei getrennte Fenster, zwei getrennte Aufgaben, keine gegenseitige Störung.
- **Mehrere Sitzungen.** Verschiedene Chats oder Projects, jeweils mit eigenem Kontext, nebeneinander.
- **Hintergrund- und geplante Aufgaben.** Arbeit, die weiterläuft, ohne dass Sie zusehen — sogar Dispatch-Aufgaben von Ihrem Handy.

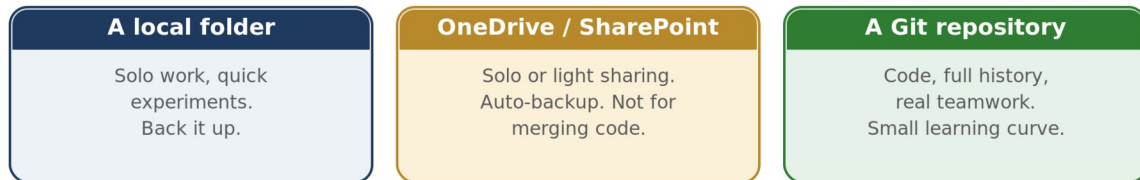
6.2 Die zwei echten Kosten

1. **Ihre Aufmerksamkeit.** Die Maschinen kommen nicht durcheinander, Sie aber schon. Jede Minute zwischen einem Dokument und einem Fehler hin- und herzuspringen, ermüdet und führt zu Fehlern. Bündeln Sie nach Kontext: eine Stunde Schreiben, dann eine Stunde an der App.
2. **Ihr Kontingent.** Zwei aufwendige Aufgaben zehren an denselben 5-Stunden- und Wochenlimits. Wenn sie gleichzeitig laufen, verbrauchen Sie Ihr Kontingent schneller, nicht langsamer.

7. Wo der Quellcode Ihres Projekts liegt

Sobald ein Projekt wichtig wird — oder mehr als eine Person daran arbeitet —, ist der Speicherort der Dateien keine Nebensache mehr.

Where your project source should live



Alone on docs → local + OneDrive. Code, or a team → Git.

7.1 Eine einfache Faustregel

- **Allein an Dokumenten?** Ein lokaler Ordner mit OneDrive-Sicherung reicht aus.
- **Allein an Code?** Nutzen Sie trotzdem Git — eine vollständige Historie und eine Rückgängig-Taste für das ganze Projekt.
- **Mit anderen an denselben Dateien?** Git für Code; eine gemeinsame SharePoint-Bibliothek (die gemeinsame Dokumentbibliothek von Microsoft) oder Drive-Bibliothek für Dokumente.

7.2 Halten Sie eine ordentliche Struktur

Was auch immer Sie wählen: Richten Sie Cowork oder Code auf einen aufgeräumten Projektordner mit klarer Gliederung aus — Quelldateien, ein Platz für Ergebnisse, eine Notiz- oder README-Datei (ein kurzer Text, der das Projekt erklärt) und nichts, was nicht dazugehört. Ein aufgeräumter Ordner macht Claude schneller und Ihre Teamkollegen gelassener.

8. Eine behutsame Einführung in Git

Git gibt einem Projekt ein Gedächtnis und ein Sicherheitsnetz. Es ist das Nützlichste, was Sie lernen können, wenn Sie von Einzelexperimenten zu echter, gemeinsamer Arbeit übergehen — und viel weniger beängstigend, als es aussieht.

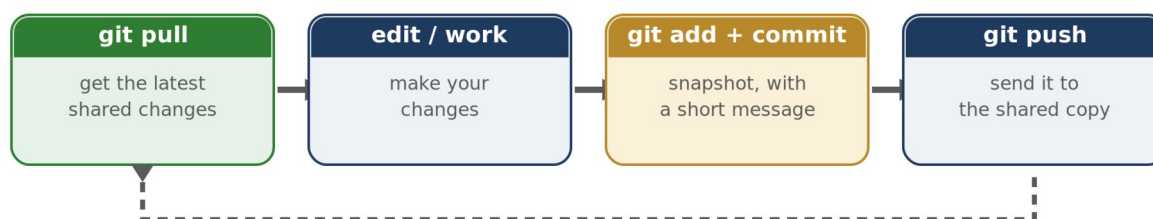
8.1 Was Git wirklich tut

Git macht Momentaufnahmen Ihres Projekts, wann immer Sie es verlangen. Jede Momentaufnahme (ein „Commit“) ist ein benannter Punkt, zu dem Sie zurückkehren können. So können Sie alles ohne Angst ändern: Wenn etwas kaputtgeht, kehren Sie zur letzten guten Momentaufnahme zurück. Nichts geht je wirklich verloren.

Begriff	Einfach gesagt
Repository (repo)	Ihr Projektordner plus seine vollständige Historie an Momentaufnahmen.
Commit	Eine gespeicherte Momentaufnahme mit einer kurzen Nachricht, was sich geändert hat.
Push / Pull	Ihre Commits an die gemeinsame Online-Kopie senden / die Commits anderer herunterladen.
Branch	Ein sicheres Nebengleis, um etwas auszuprobieren, ohne die Hauptversion anzutasten.
Clone	Eine eigene lokale Kopie eines gemeinsamen Repos anlegen.

8.2 Der tägliche Ablauf

The everyday Git loop — four steps, repeated



You can just tell Claude in Code: **“commit my changes with a sensible message and push.”**

GitHub (oder GitLab oder — in einer Microsoft-Umgebung — Azure DevOps) ist die Website, auf der die gemeinsame Kopie liegt. Sie können Claude in Code bitten, die Schritte für Sie auszuführen — „Committe meine Änderungen mit einer sinnvollen Nachricht und mach einen Push“ –, aber es hilft trotzdem zu wissen, was sie bedeuten.

8.3 Workshop — Ihr erstes Repo in 10 Minuten

1. Öffnen Sie in Code einen Projektordner und sagen Sie: „Richte diesen Ordner als Git-Repository ein.“
2. Machen Sie eine kleine Änderung und sagen Sie dann: „Committe sie mit einer klaren Nachricht.“
3. Legen Sie ein kostenloses GitHub-Konto an und sagen Sie: „Erstelle ein privates Repo und push das dorthin.“

4. Ändern Sie wieder etwas, machen Sie einen Commit und einen Push — und sehen Sie zu, wie die Historie online wächst.

Zwei Regeln, die den meisten Ärger mit Git verhindern

- 1) Machen Sie zuerst einen Pull, bevor Sie anfangen, und committen Sie oft in kleinen Schritten. Kleine Schnappschüsse sind leicht zu verstehen und rückgängig zu machen.
- 2) Legen Sie nie Geheimnisse (API-Schlüssel, Passwörter) in ein Repo. Sobald sie gepusht sind, gelten sie als öffentlich. Bewahren Sie sie in einer separaten lokalen Datei auf, die Git ignorieren soll.

9. Im Team am selben Projekt arbeiten

Sobald zwei Personen dieselben Dateien bearbeiten, brauchen Sie einen Weg, ihre Arbeit zusammenzuführen, ohne dass einer die Arbeit des anderen überschreibt.

9.1 Dokumente und Code

- **Gemeinsame Dokumente** (Word, Excel, Folien): Eine gemeinsame SharePoint- oder Drive-Bibliothek verwaltet Versionen und zeigt, wer was geändert hat. Vermeiden Sie, dass zwei Personen genau dieselbe Datei im selben Moment bearbeiten.
- **Gemeinsamer Code** (Skripte, Apps, alles Textbasierte): Nutzen Sie Git. Git ist genau dafür gebaut, Änderungen mehrerer Personen sicher zusammenzuführen.

9.2 Der Git-Ablauf im Team

1. **Zuerst ein Pull**, damit Sie auf der Arbeit der anderen aufbauen.
2. **Arbeiten Sie auf einem Branch**, der nach der Aufgabe benannt ist, nicht auf der Hauptversion.
3. **Öffnen Sie einen Pull Request** – ein Teammitglied schaut kurz darüber, dann kommt er in die Hauptversion.

Dieser Prüfschritt ist keine Bürokratie. Er ist ein zweites Paar Augen, und dort entsteht gemeinsames Verständnis. Wenn zwei Personen dieselben Zeilen ändern, fragt Git, welche Version bleiben soll (ein „merge conflict“). Das klingt beunruhigend, ist aber Routine. Sie können Claude in Code bitten, ihn zu erklären und zu lösen.

10. Ein behutsamer erster Schritt in Code

Claude Code schreibt und startet echte Software. Sie müssen den Code nicht selbst lesen oder schreiben. Sie beschreiben in einfachen Worten, was Sie wollen, und Claude setzt es um.

10.1 Beschreiben statt tippen

„Füg eine Schaltfläche hinzu, die die Tabelle nach Excel exportiert“ ist eine vollständige Anweisung. Claude schlägt die Änderung vor, Sie genehmigen sie, und sie wird ausgeführt. Sie bleiben der Entscheider; Claude übernimmt das Tippen.

10.2 CLAUDE.md — das Gehirn des Projekts

Jedes Code-Projekt sollte im obersten Ordner eine Datei namens `CLAUDE.md` haben. Sie wird zu Beginn jeder Sitzung gelesen. Sie enthält, wie das Projekt aufgebaut ist, wie man es startet, welche Entscheidungen Sie getroffen haben und was Sie bevorzugen. Sagen Sie nach einer guten ersten Sitzung: „Schreib eine `CLAUDE.md` mit allem, was du über dieses Projekt gelernt hast.“ Halten Sie sie aktuell.

10.3 Berechtigungen — drei Stufen, nicht zwei

- **Jedes Mal fragen** (Standard): sicher, langsam.
- **Änderungen automatisch annehmen**, aber riskante Befehle bestätigen: ein guter Mittelweg.
- **Alle Berechtigungen umgehen** (schneller „Vibe“-Modus): nur in einem eigenen, mit Git gesicherten Ordner, dessen Dateien Sie ersetzen könnten. Nie in Ihrem Home-Verzeichnis (Ihrem gesamten persönlichen Benutzerordner).

10.4 Erst planen, dann bauen

Bitte Sie Claude bei allem, was nicht trivial ist, zuerst zu untersuchen und einen kurzen Plan vorzuschlagen, bevor eine Zeile geschrieben wird: „Zeig mir zwei Optionen und warte auf meine Wahl.“ Wenn Sie einen Plan vorab genehmigen, vermeiden Sie lange Umwege und behalten die Kontrolle.

10.5 Vibe coding — und seine Grenzen

Was Sie hier tun, hat einen Namen: vibe coding. Sie bauen Software, indem Sie beschreiben, was Sie wollen, und nach Gefühl steuern, statt den Code zu lesen und zu schreiben. Das funktioniert hervorragend für persönliche Werkzeuge, Prototypen und Wegwerf-Skripte. Für alles, worauf sich andere Menschen verlassen, gelten strengere Regeln: Halten Sie es in Git, starten und testen Sie wirklich, was dabei herauskommt, und geben Sie nie etwas heraus, das Sie nicht prüfen können. Nutzen Sie vibe coding frei bei Dingen mit wenig Risiko. Werden Sie langsamer und prüfen Sie bei allem, was wichtig ist.

10.6 Claude Design — Oberflächen ohne Code

Claude Design ist ein Verwandter von Code und startete 2026 als Research Preview (eine frühe Testversion) unter claude.ai/design. Claude Design baut das Aussehen von Dingen aus einer einfachen Beschreibung: Mockups (erste Entwürfe eines Bildschirms), Landingpages, einfache Websites und Foliensätze. Anders als ein Bildgenerator liefert es das echte Ergebnis: ein funktionierendes Layout mit Schaltflächen, die Sie wirklich anklicken können, kein Bild davon. Greifen Sie dazu, wenn Sie eine Seite oder ein Design wollen und keine Software. Es ist in Pro, Max, Team und Enterprise enthalten. Da es eine Research Preview ist, wird es sich weiter verändern.

11. Ihr erster kleiner Agent

Ein Agent ist nichts anderes als Claude, der über mehrere Schritte auf ein Ziel hinarbeitet, oft nach Zeitplan, und dann berichtet. Sie brauchen kein großes System. Eine einzige nützliche, wiederkehrende Aufgabe ist schon ein echter Agent.

11.1 Was einen Agenten ausmacht

Drei Zutaten: eine klare Aufgabe, ein Auslöser (eine Uhrzeit oder ein Ereignis) und ein Weg, Ihnen das Ergebnis mitzuteilen. Alles andere sind Details.

11.2 Ein vollständiges Beispiel — ein Morgenbriefing

1. **Legen Sie die Aufgabe fest:** „Liste mir an jedem Werktag um 7 Uhr meine heutigen Termine und alle ungelesenen, als wichtig markierten Mails auf, und schreib eine Zusammenfassung in 5 Zeilen.“
2. **Machen Sie daraus eine gespeicherte Aufgabe,** damit sie jedes Mal gleich läuft, mit den Mail- und Kalender-Connectors nur im Lesemodus.
3. **Planen Sie sie** für 07:00 Uhr an Werktagen ein (Ihr Rechner oder ein ständig laufender Rechner muss eingeschaltet sein).
4. **Lassen Sie sie dorthin senden, wo Sie sie sehen:** eine E-Mail an sich selbst oder eine Nachricht auf Ihr Handy.

Beginnen Sie mit einem Probelauf

Starten Sie die Aufgabe einige Male von Hand und lesen Sie das Ergebnis kritisch, bevor Sie sie einplanen. Ein Agent, der jeden Morgen einen kleinen Fehler wiederholt, ist schlechter als gar kein Agent.

11.3 Langsam ausbauen

Fügen Sie eine Fähigkeit nach der anderen hinzu. Führen Sie eine kurze Notizdatei darüber, was der Agent tut und wie man ihn repariert. Nur weil Claude in Sekunden eine weitere Funktion hinzufügen kann, heißt das nicht, dass es sie geben sollte.

12. Werkzeuge und Apps zum Installieren

Mit der Desktop-App allein kommen Sie weit. Aber eine Handvoll kostenloser Standardwerkzeuge macht das Bauen angenehmer. Installieren Sie sie, wenn Sie sie brauchen, nicht alle auf einmal.

Werkzeug	Wofür es dient	Wann Sie es brauchen
Ein Code-Editor (VS Code oder Notepad++)	Text- und Code-Dateien lesen und leicht bearbeiten.	Sobald Sie die Dateien eines Projekts selbst öffnen.
Git + ein GitHub-Konto	Versionsverlauf und Teamarbeit (Kapitel 8–9).	Sobald ein Projekt wichtig ist oder geteilt wird.
Python und Node.js	Die Motoren, auf denen die meisten kleinen Werkzeuge und Skripte laufen.	Wenn Code etwas baut, das Sie lokal starten möchten.
PuTTY / WinSCP	Verbindung zu einem Server oder einem Raspberry Pi (ein kleiner, günstiger Computer) herstellen und Dateien dorthin kopieren.	Wenn Sie Aufgaben auf einem ständig laufenden Rechner ausführen.
Ein Messaging-Bot (z. B. Telegram)	Damit ein Agent Ihnen Warnungen und Berichte senden kann.	Wenn Sie Ergebnisse direkt auf Ihr Handy bekommen möchten.

Lassen Sie Claude die Einrichtung übernehmen

Sagen Sie: „Führ mich Schritt für Schritt durch die Installation von Git unter Windows und prüf jeden Schritt mit mir.“ Das funktioniert genauso für Python, Node oder die Verbindung zu einem Pi.

13. Wartbar bleiben

Der Unterschied zwischen einem lustigen Wochenende und einem System, dem Sie auch in sechs Monaten noch vertrauen, sind ein paar stille Gewohnheiten.

- **Sichern Sie an zwei Orten.** Quellen in Git, wichtige Ordner synchronisiert oder gezippt. Eine einzige Kopie ist keine Sicherung.
- **Committen und notieren Sie laufend.** Kleine Commits und eine kurze README sorgen dafür, dass Ihr zukünftiges Ich ein Projekt schnell versteht.
- **Nutzen Sie Gedächtnis und Sitzungsnotizen im großen Stil.** Eine MEMORY.md und Notizen pro Projekt geben jeder neuen Sitzung Orientierung.
- **Räumen Sie Skills und Connectors auf,** die Sie nicht mehr nutzen. Verwaiste sind Ballast und ein Risiko.
- **Wissen Sie, wann Schluss ist.** Fragen Sie vor jeder neuen Funktion, ob sie ihren Platz verdient. Weglassen ist eine Fähigkeit.

Der Ein-Satz-Test

Bei jedem Projekt sollten Sie in einem Satz sagen können, was es tut und warum es existiert. Wenn Sie das nicht können, ist es ohne Richtung gewachsen. Räumen Sie es auf, bevor Sie es erweitern.

Glossar der neuen Begriffe

Prompts und Projects

Begriff	In einfachen Worten
Anti-Stil-Datei	Eine kurze Liste von Wörtern und Mustern, die Claude nie verwenden soll.
Project-Anweisungen	Fünf bis acht Zeilen, die Claude sagen, wofür ein Project da ist und wie es arbeiten soll.
Skill	Eine wiederverwendbare Anweisungsdatei (SKILL.md), die Claude eine Aufgabe beibringt.
Record a skill	Zeigen Sie Claude eine Aufgabe einmal auf dem Bildschirm; Claude macht aus der Vorführung einen Skill.

Verbinden und Daten

Begriff	In einfachen Worten
Connector	Eine sichere Brücke zwischen Claude und einer anderen App, etwa Mail oder Kalender.
MCP	Der gemeinsame Standard (Model Context Protocol), den Connectors nutzen.
Erstanbieter / Drittanbieter	Von Anthropic selbst entwickelt / von einem anderen Unternehmen entwickelt.
Nur Lesen	Claude kann nachsehen, aber nichts senden oder ändern.
DSGVO	Die EU-Regeln für den Umgang mit personenbezogenen Daten: alles, was eine lebende Person identifiziert.
Anonymisieren / schwärzen	Namen und Details entfernen, die Personen identifizieren.
Datenschutzbeauftragte(r)	Die Person in Ihrer Organisation, die für personenbezogene Daten zuständig ist.
Aufbewahrung	Wie lange Ihre Daten gespeichert werden.

Git und Teamarbeit

Begriff	In einfachen Worten
Repository (Repo)	Ein Projektordner mit dem vollständigen Verlauf seiner gespeicherten Schnappschüsse, verwaltet von Git.
Commit	Ein gespeicherter Schnappschuss mit einer kurzen Nachricht, die die Änderung beschreibt.
Push / Pull	Ihre Commits an die gemeinsame Online-Kopie senden / die der anderen zu Ihrer Kopie holen.
Branch	Ein sicheres Nebengleis, um Änderungen zu testen, ohne die Hauptversion anzutasten.
Pull Request	Eine Anfrage, Ihren Branch in die Hauptversion zu übernehmen, meist nach vorheriger Prüfung.

Begriff	In einfachen Worten
Merge conflict	Zwei Personen haben dieselben Zeilen geändert; Git fragt, welche Version bleiben soll. Routine.
Clone	Eine eigene lokale Kopie eines gemeinsamen Repos anlegen.
GitHub / GitLab	Websites, die die gemeinsame Kopie eines Git-Repos bereitstellen, damit ein Team sie teilen kann.

Code und Agenten

Begriff	In einfachen Worten
Claude Code	Claude, der echte Software auf Ihrem PC schreibt und startet.
CLAUDE.md	Eine Datei im obersten Ordner eines Code-Projekts, die Claude erklärt, wie das Projekt funktioniert.
Berechtigungen	Wie viel Claude tun darf, ohne zu fragen: jedes Mal fragen, Änderungen automatisch annehmen oder alles umgehen.
Vibe coding	Software bauen, indem Sie beschreiben, was Sie wollen, statt den Code zu schreiben.
Claude Design	Baut Mockups, Landingpages und einfache Websites aus einer einfachen Beschreibung.
Agent	Claude, der über mehrere Schritte auf ein Ziel hinarbeitet, oft nach Zeitplan, und dann berichtet.
Auslöser	Die Uhrzeit oder das Ereignis, das einen Agenten startet.
API	Ein Weg, wie Ihr eigenes Programm direkt mit Claude spricht, abgerechnet pro Token.

Wie es weitergeht

Wenn Sie Stufe Null und dieses Buch zu Stufe Eins durchgearbeitet haben, können Sie Claude jetzt gut einrichten, echte Ergebnisse erstellen, verantwortungsvoll mit Daten umgehen, Ihre Quellen sicher aufbewahren und teilen, einen ersten Schritt in Code und Claude Design machen und einen kleinen Agenten betreiben. Nach meiner Erfahrung deckt das das meiste ab, was Menschen im Alltag brauchen.

Die API — der Schritt über dieses Buch hinaus

Vielleicht haben Sie schon von der **Claude API** gehört (Teil der Claude Platform). Die App, Cwork und Code lassen Sie Claude über ein Fenster nutzen, das jemand anderes gebaut hat. Mit der API spricht ein eigenes Programm direkt mit Claude, abgerechnet pro Token. So können Sie eigene Apps und Agenten bauen. Das ist technischer und liegt deshalb jenseits dieses Buchs. Aber es ist näher, als es klingt: Sobald ein kleines Werkzeug, das Sie in Code bauen, Claude selbstständig aufruft, nutzt es bereits die API.

Und behalten Sie die zwei Gewohnheiten aus Stufe Null im Blick: **Lassen Sie Claude zuerst fragen, und vertrauen Sie nie einer selbstsicheren Antwort, ohne sie zu prüfen.** Sie werden wichtiger, nicht unwichtiger, je größer das wird, was Sie bauen.

Von Hans Tandt